CMPSC 461 – Fall 2025 Homework

Assignment 3

Name: Mohammad Abu Shariha
User ID: mma7020

Problem 1: Storage, Scope, and Lifetime

(A) Storage Allocation

- A (`const int A=100`) → static object (global const).
- B (parameter in `process`) → stack.
- C (local in `process`) → stack.
- D (`static int D=0` in main) → static object.
- E (`int E=20`) → stack.
- F (pointer itself) → stack.
- The `new int(50)` → heap.
- G (inside if) → stack.

(B) Scope and Lifetime

So basically, scope is where you can see the variable, and lifetime is how long it exists in memory:

- A: scope is global, lifetime is whole program.
- B: only visible in process, gone after the call.
- C: same as B.
- D: scope is only inside main, but lifetime is whole program because it's static.
- E and F: local to main, only while main runs.
- The heap int: lives until delete is called.
- G: only inside that if-block.

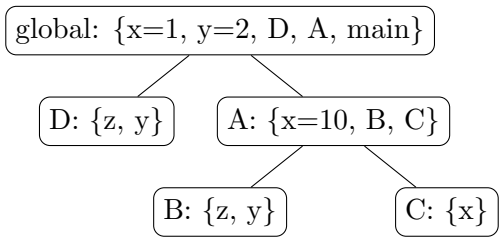
(C) Scope vs Lifetime of D

D is a good example: I can only use it inside main (scope), but it's actually around until the program ends (lifetime).

Problem 2: Static Scoping and Symbol Tables

(A) Symbol Table Hierarchy

Under static scoping, the nesting at compile-time matters, not who calls who. Here's the lexical tree:



(B) Program Output

Step by step:

1. C sets local x = y + 5 = 2 + 5 = 7 (since it finds y=2 in global).
2. D(7): local y=14. Prints y + global x = 14 + 1 = 15.
3. Back in A: print x=10.
4. B(5): y = 5 + 10 = 15 (x=10 from A). Prints 15.
5. A again prints x=10.
6. main prints y=2 (global).

Final output:

```
15
10
15
10
2
```

(C) Scope Resolution

Here's how each lookup resolves using the symbol table:

- In A: x is local (10). y not found in A, so it's global (2).
- In B: y is local (z+10). x not in B, so found in A (10).
- In C: local x (7). y not in C, not in A, found in global (2).
- In D: y is local. x not in D, so global (1).
- In main: y is global (2).

Problem 3: Nested Scopes, Recursion, and Links

(A) Static Scoping Output

Trace:

- helper() sees x from global, which is 100.
- outer() sets x=50 and then prints it.
- Finally main prints its global x=100.

So output:

```
100
50
100
```

(B) Dynamic Scoping Output

Trace:

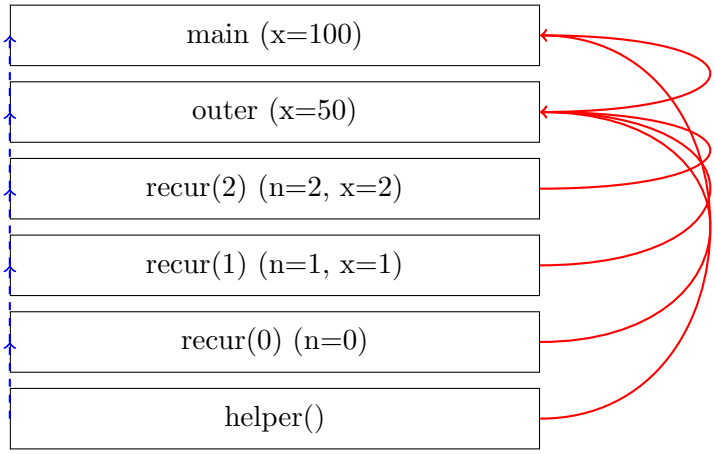
- helper() follows dynamic chain to recur(1), finds x=1.
- outer() has x=50, prints 50.
- main prints its global x=100.

So output:

```
1
50
100
```

(C) Runtime Stack at helper()

The call stack when helper runs:



Red = static link, Blue dashed = dynamic chain

(D) Binding

So basically:

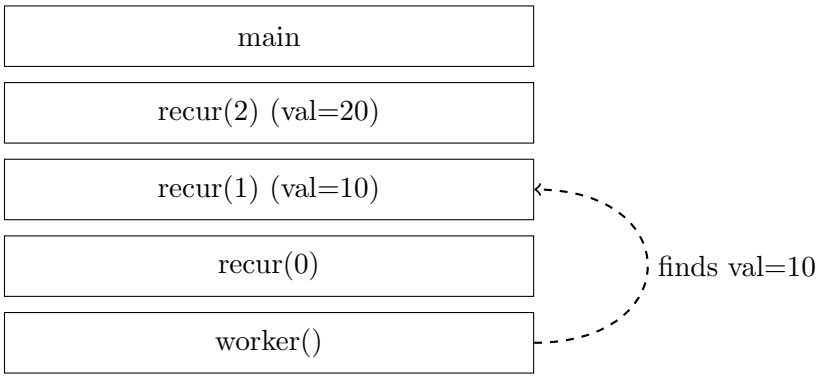
- Static scoping: helper is defined globally, so its static link goes straight to global scope. It finds global x=100.
- Dynamic scoping: helper walks the dynamic chain, finds x=1 from recur(1).

Problem 4: Deep vs Shallow Binding and Closures

(A) Shallow Binding

Shallow means look up when called. Closest active val=10. Output:

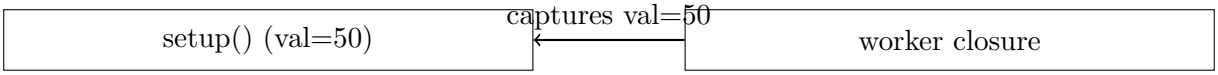
```
10
```



(B) Deep Binding

Deep means it remembers the environment when created. At that time val=50. Output:

```
50
```



(C) Closure

A closure = function + the environment it was created in. Here, worker keeps val from setup even after setup ends. That only works with deep binding.